Paradigm Leveling Model Comparison

```
library(dplyr)
library(tidyr)
library(brms)
library(rstan)
library(loo)
library(bayesplot)
library(ggplot2)
library(tidybayes)
library(marginaleffects)
library(modelr)
library(purrr)

options(mc.cores = 6)
```

Data loading

Could be useful for inspection:

```
model_data <- read.csv("data_for_analysis.csv")
head(model_data)
```

```
##      lemma lemma_std date   id        variety std_infl   log_freq is_bipartite
## 1 sprëchen        27 1100 M002 Central German   PastSg  1.4977261            0
## 2    lâzen         4 1150 M004   Upper German   PastSg  0.3676274            0
## 3 sprëchen        27 1150 M004   Upper German   PastSg -3.5477195            0
## 4    hèben        90 1150 M004   Upper German   PastSg -1.3671875            0
## 5   fahren        89 1150 M004   Upper German   PastSg -0.1644030            0
## 6    sëhen        22 1150 M004   Upper German   PastSg -0.8339498            0
##   element_type has_levelled
## 1        vowel            0
## 2        vowel            0
## 3        vowel            0
## 4        vowel            0
## 5        vowel            0
## 6        vowel            0
```

Model loading and comparison

```
# Load the three new models
fit_base   <- readRDS("models/base_fit.rds")
fit_k10    <- readRDS("models/base_fit_k10.rds")
```

```r
fit_tensor <- readRDS("models/base_fit_tensor_product.rds")

# Put them in a named list for easier iteration
models <- list(
  "Base (k=4)" = fit_base,
  "Base (k=10)" = fit_k10,
  "Tensor Product" = fit_tensor
)
```

```r
# Compute LOO for each model and compare
# Warning: This step might take a few moments depending on model size.
loo_list <- map(models, loo)
comparison <- loo_compare(fit_base, fit_k10, fit_tensor)

print(comparison, simplify = FALSE)
```

```
##            elpd_diff se_diff elpd_loo se_elpd_loo p_loo   se_p_loo looic
## fit_k10        0.0       0.0 -1011.9     46.3       103.1     6.6   2023.8
## fit_tensor   -25.6       5.9 -1037.5     46.4       114.9     7.0   2075.0
## fit_base     -26.0       6.0 -1037.9     46.5       121.0     7.3   2075.9
##            se_looic
## fit_k10       92.6
## fit_tensor    92.8
## fit_base      93.0
```

```r
stacking_weights <- loo_model_weights(loo_list, method = "stacking")
print(stacking_weights)
```

```
## Method: stacking
## ------
##                 weight
## Base (k=4)     0.000
## Base (k=10)    1.000
## Tensor Product 0.000
```

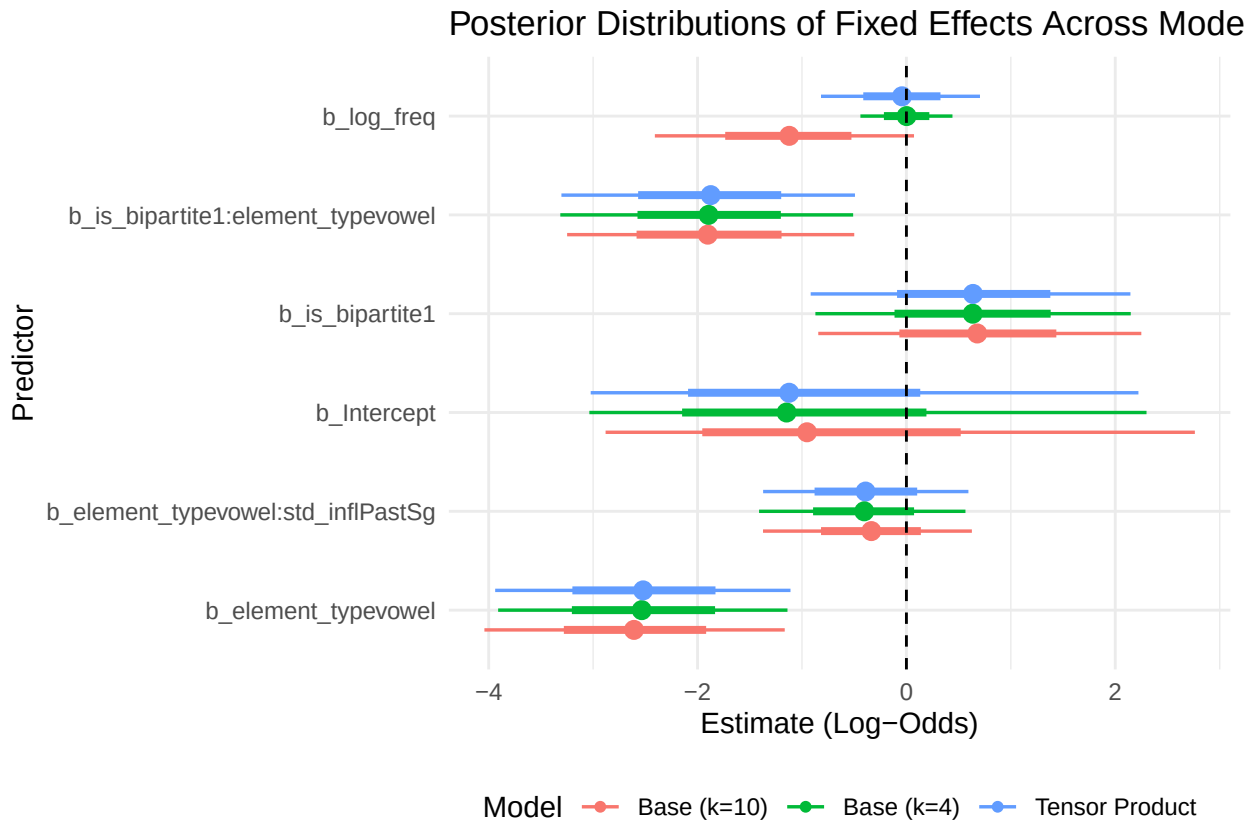
Fixed effects

```r
# Extract fixed effects draws for all models
extract_fixed_draws <- function(model, model_name) {
  model %>%
    gather_draws(`^b_[^s].*`, regex = TRUE) %>% # Exclude spline terms
    mutate(Model = model_name)
}

all_fixed_draws <- map2_dfr(models, names(models), extract_fixed_draws)

ggplot(all_fixed_draws, aes(x = .value, y = .variable, color = Model)) +
  stat_pointinterval(position = position_dodge(width = 0.6)) +
  geom_vline(xintercept = 0, linetype = "dashed", color = "black") +
```

```
  labs(
    title = "Posterior Distributions of Fixed Effects Across Models",
    x = "Estimate (Log-Odds)",
    y = "Predictor"
  ) +
  theme_minimal() +
  theme(legend.position = "bottom")
```



Effect of bipartite marking

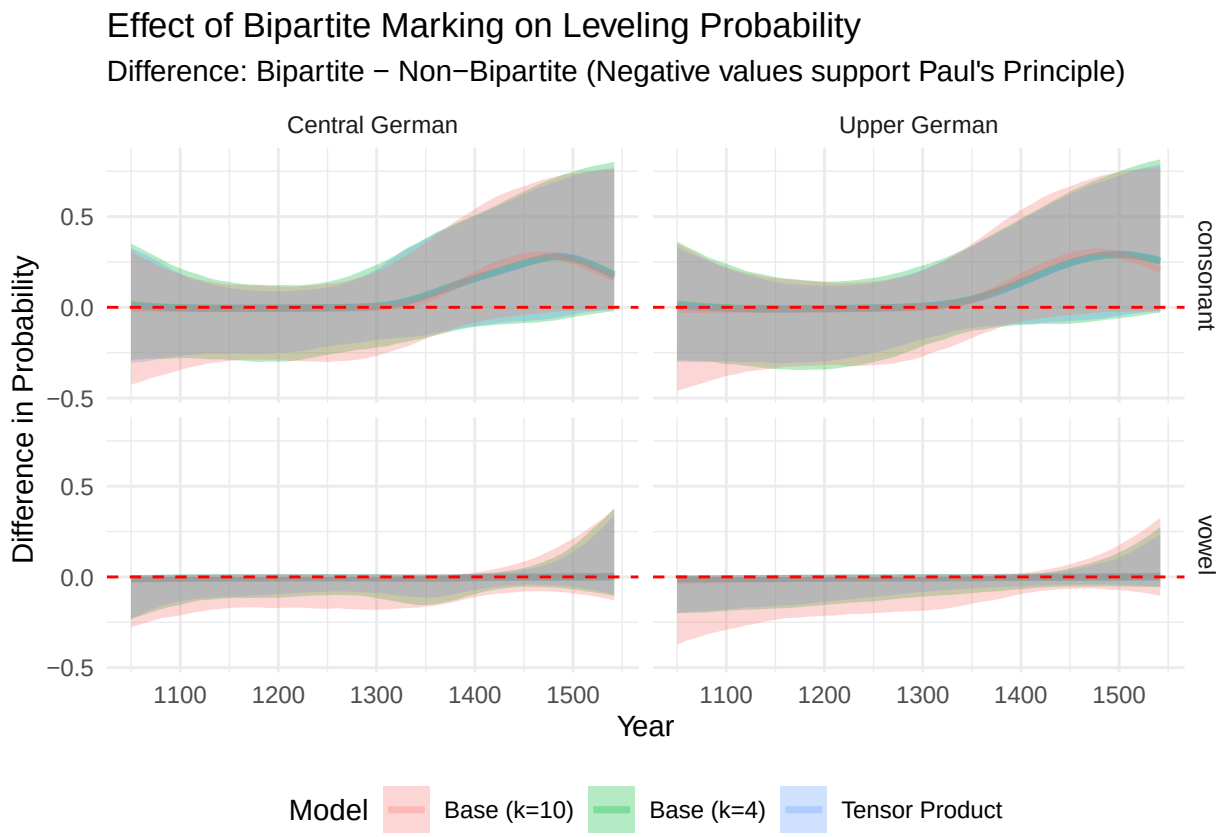
```
# Generate prediction grid
grid_full <- model_data %>%
  data_grid(
    date = seq_range(date, n = 50),
    is_bipartite = c("0", "1"),
    element_type = c("vowel", "consonant"),
    variety = c("Central German", "Upper German"),
    std_infl = "PastSg", # Holding constant for the plot
    log_freq = mean(model_data$log_freq, na.rm = TRUE)
  )

# Function to get predictions and calculate difference
```

```r
get_bipartite_diff <- function(model, model_name) {
  grid_full %>%
    add_epred_draws(model, re_formula = NA) %>%
    ungroup() %>%
    select(date, is_bipartite, element_type, variety, .epred, .draw) %>%
    pivot_wider(names_from = is_bipartite, values_from = .epred) %>%
    mutate(
      diff = `1` - `0`,
      Model = model_name
    )
}

# Apply to all models and combine
plot_data_diff <- map2_dfr(models, names(models), get_bipartite_diff)

ggplot(plot_data_diff, aes(x = date, y = diff, fill = Model, color = Model)) +
  stat_lineribbon(.width = 0.95, alpha = 0.3) +
  geom_hline(yintercept = 0, linetype = "dashed", color = "red") +
  facet_grid(element_type ~ variety) +
  labs(
    title = "Effect of Bipartite Marking on Leveling Probability",
    subtitle = "Difference: Bipartite - Non-Bipartite (Negative values support Paul's Principle)",
    x = "Year",
    y = "Difference in Probability"
  ) +
  theme_minimal() +
  theme(legend.position = "bottom")
```



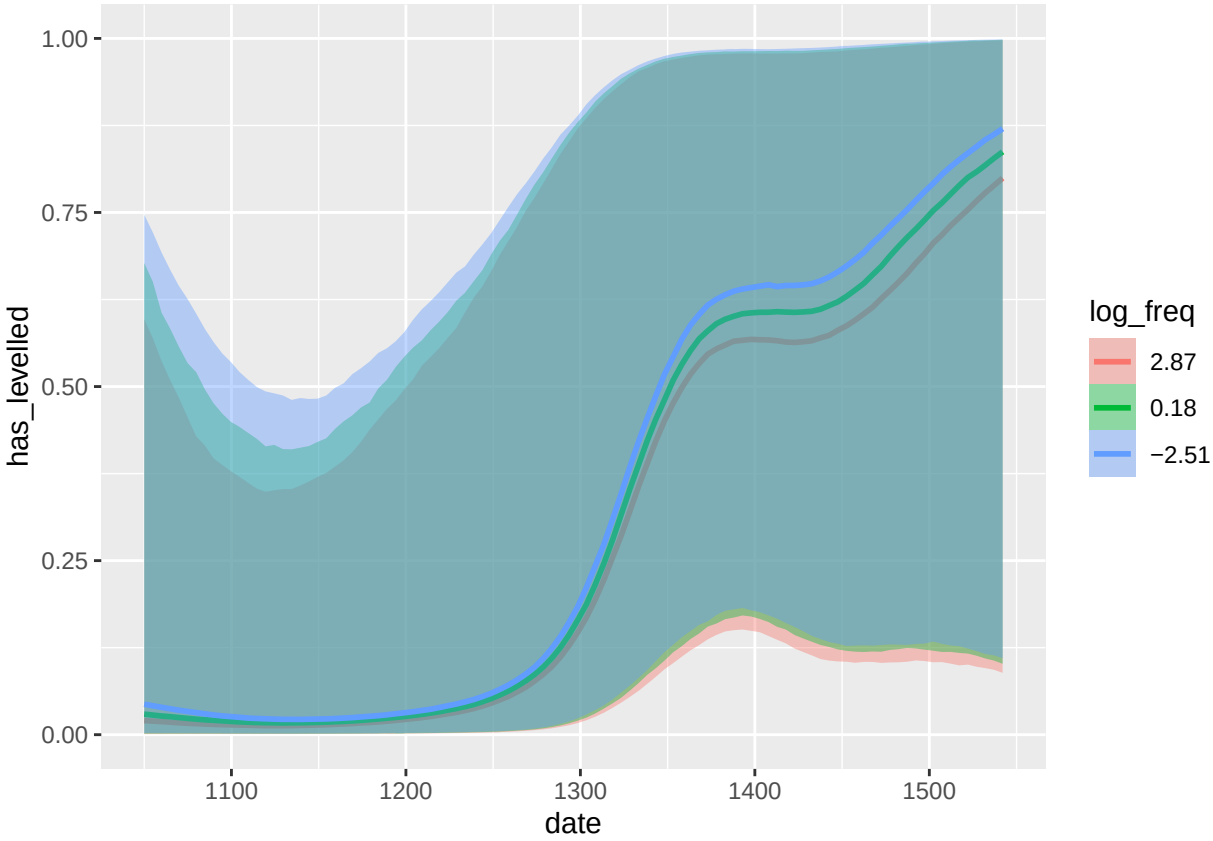
And now marginal effect

```
ame_bipartite <- avg_comparisons(
  fit_k10,
  variables = "is_bipartite",
  ndraws = 500
)
print(ame_bipartite)
```

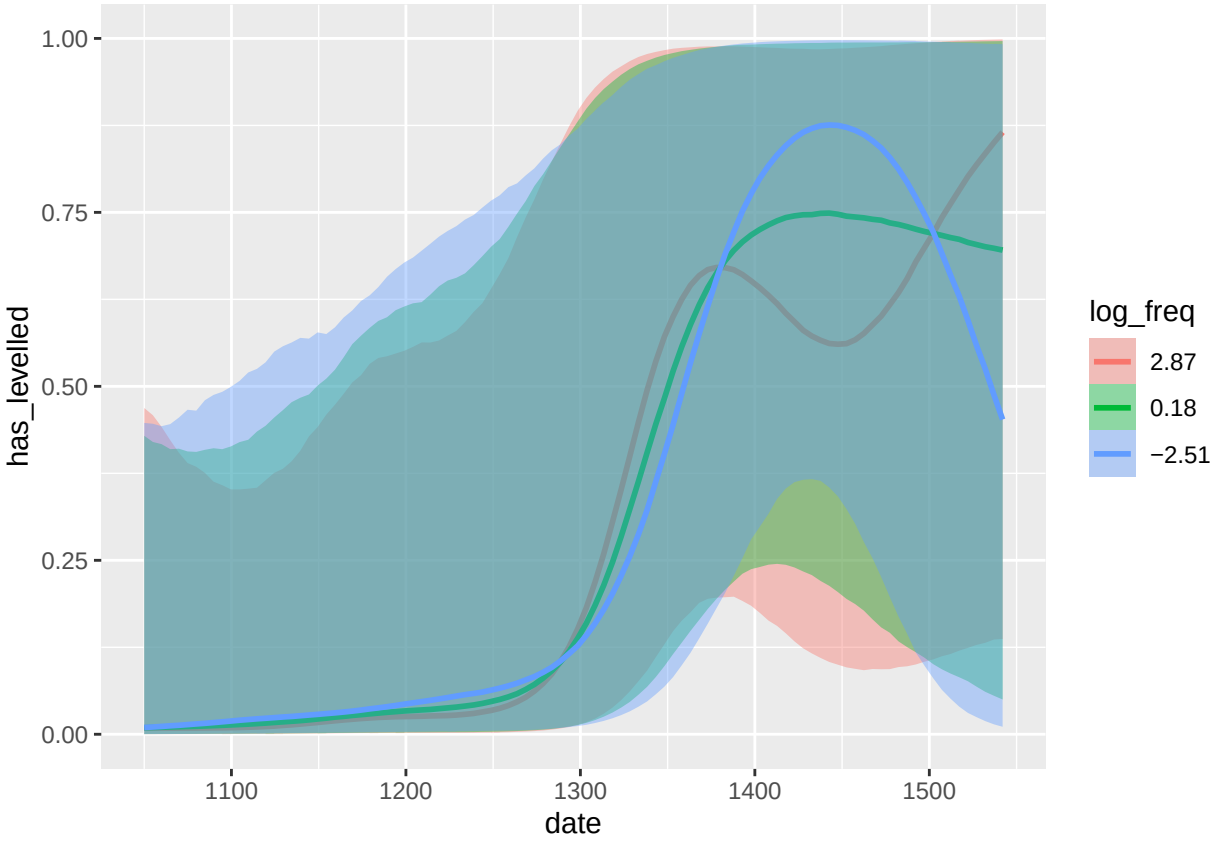
```
##
##  Estimate   2.5 % 97.5 %
##   -0.0126 -0.0256 0.0174
##
## Term: is_bipartite
## Type: response
## Comparison: 1 - 0
```

Time and frequency effect

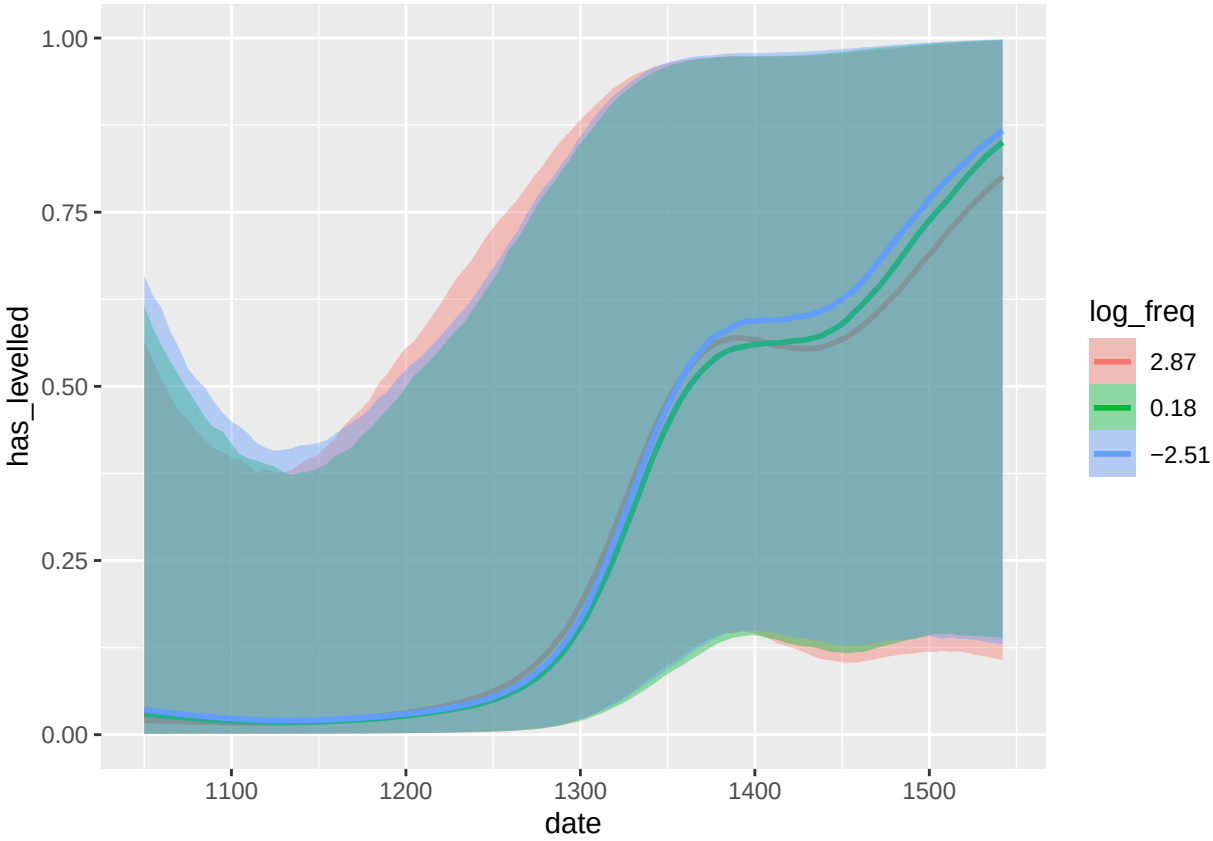
```
# This will output a line plot with 3 different trajectories representing
# average, low, and high frequency verbs over time.
conditional_effects(fit_base, effects = "date:log_freq")
```



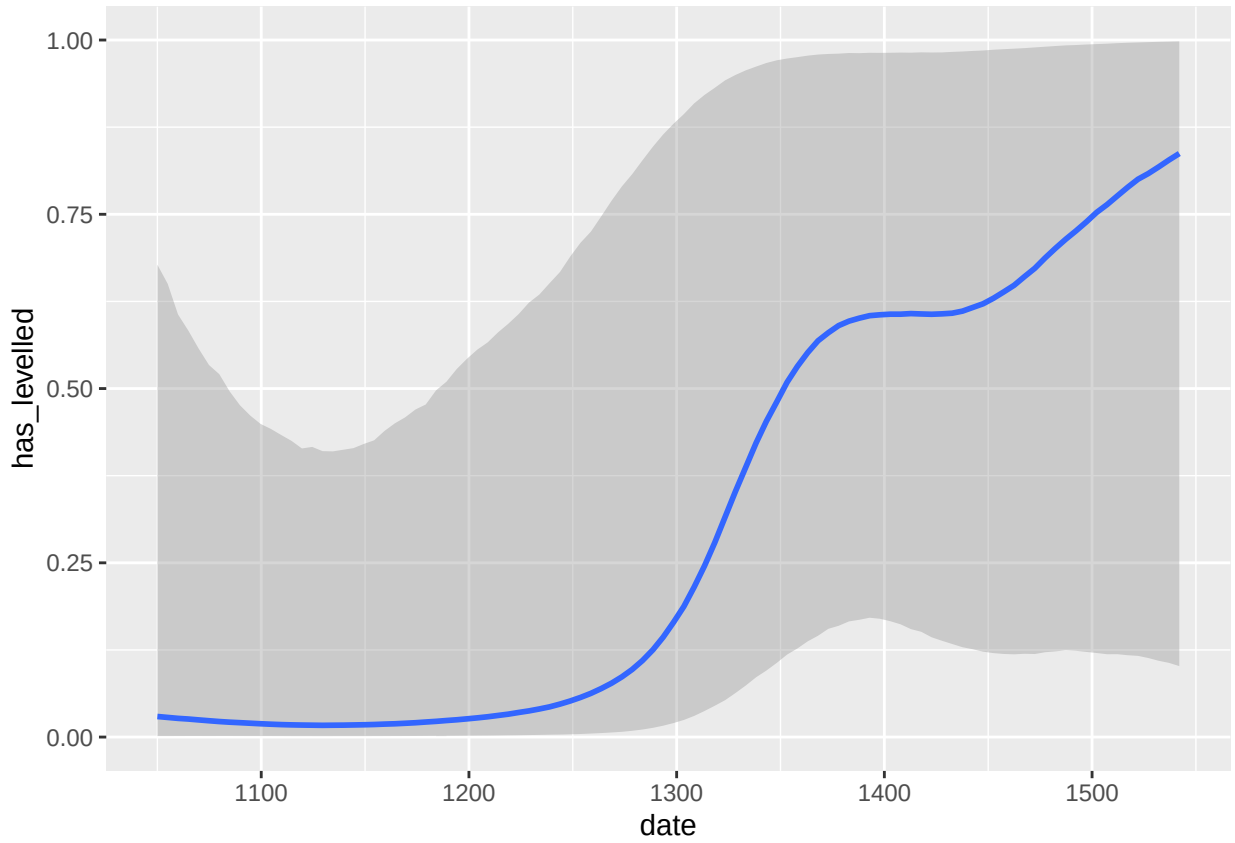

```
conditional_effects(fit_k10, effects = "date:log_freq")
```



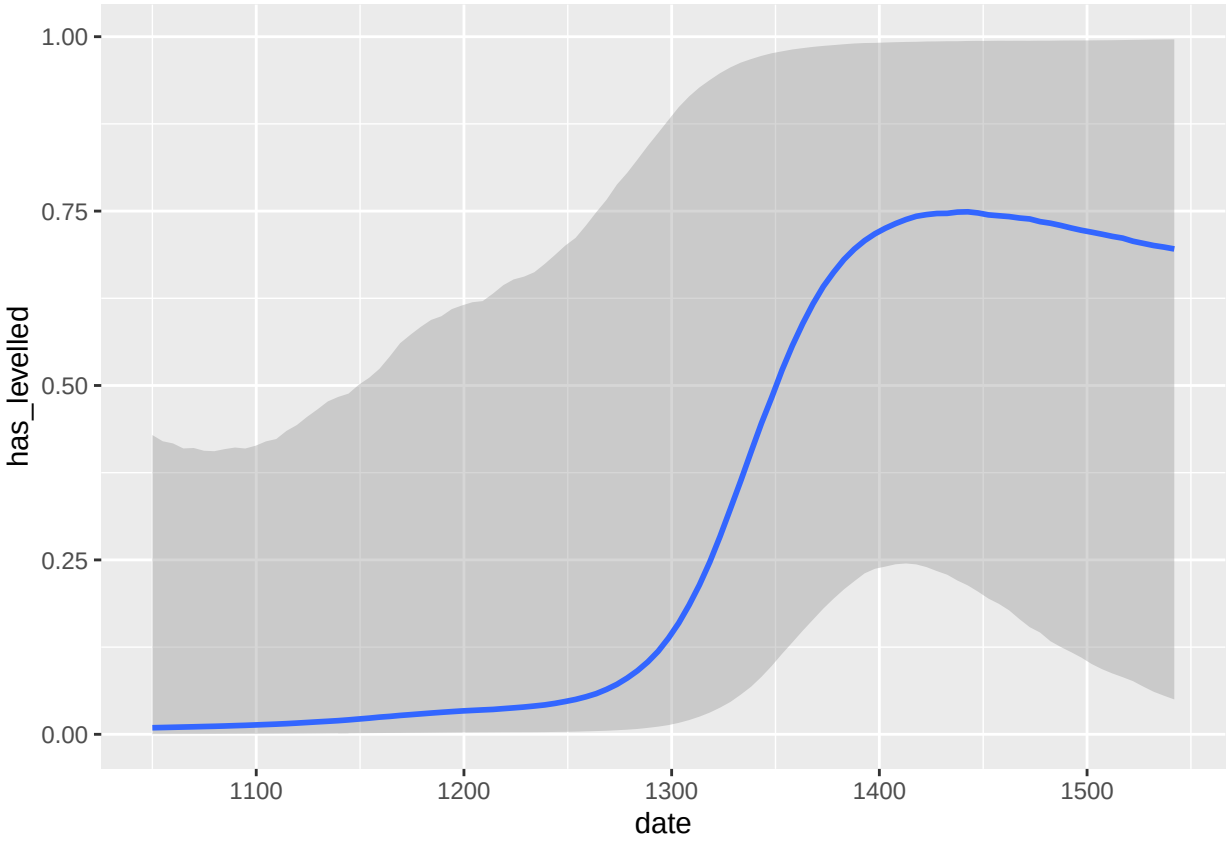

```
conditional_effects(fit_tensor, effects = "date:log_freq")
```



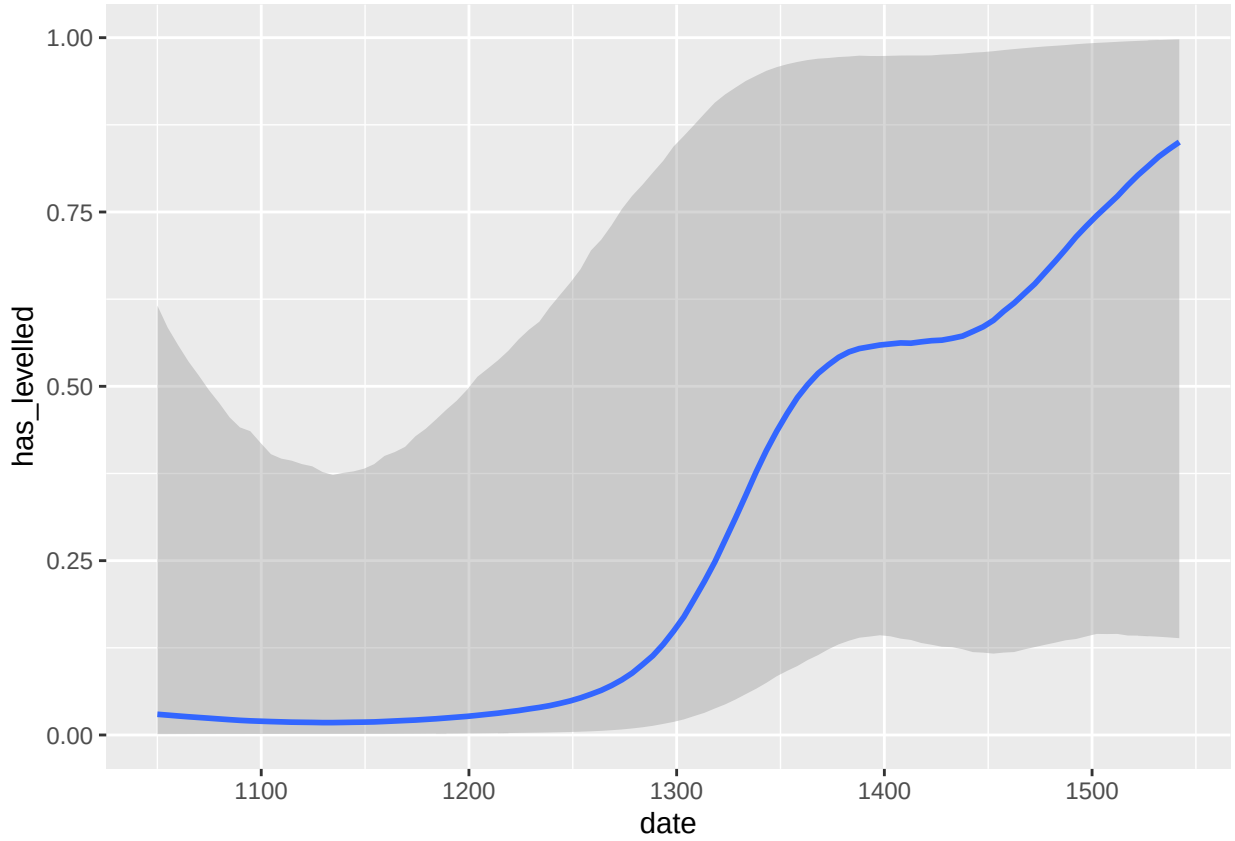

```
conditional_effects(fit_base, effects = "date")
```




```
conditional_effects(fit_k10, effects = "date")
```




```
conditional_effects(fit_tensor, effects = "date")
```



Marginal effect of frequency

```
ame_freq <- avg_slopes(
  fit_k10,
  variables = "log_freq",
  ndraws = 500
)

print(ame_freq)
```

```
##
##  Estimate    2.5 %   97.5 %
##  -0.00166 -0.00346 1.74e-05
##
## Term: log_freq
## Type: response
## Comparison: dY/dX
```

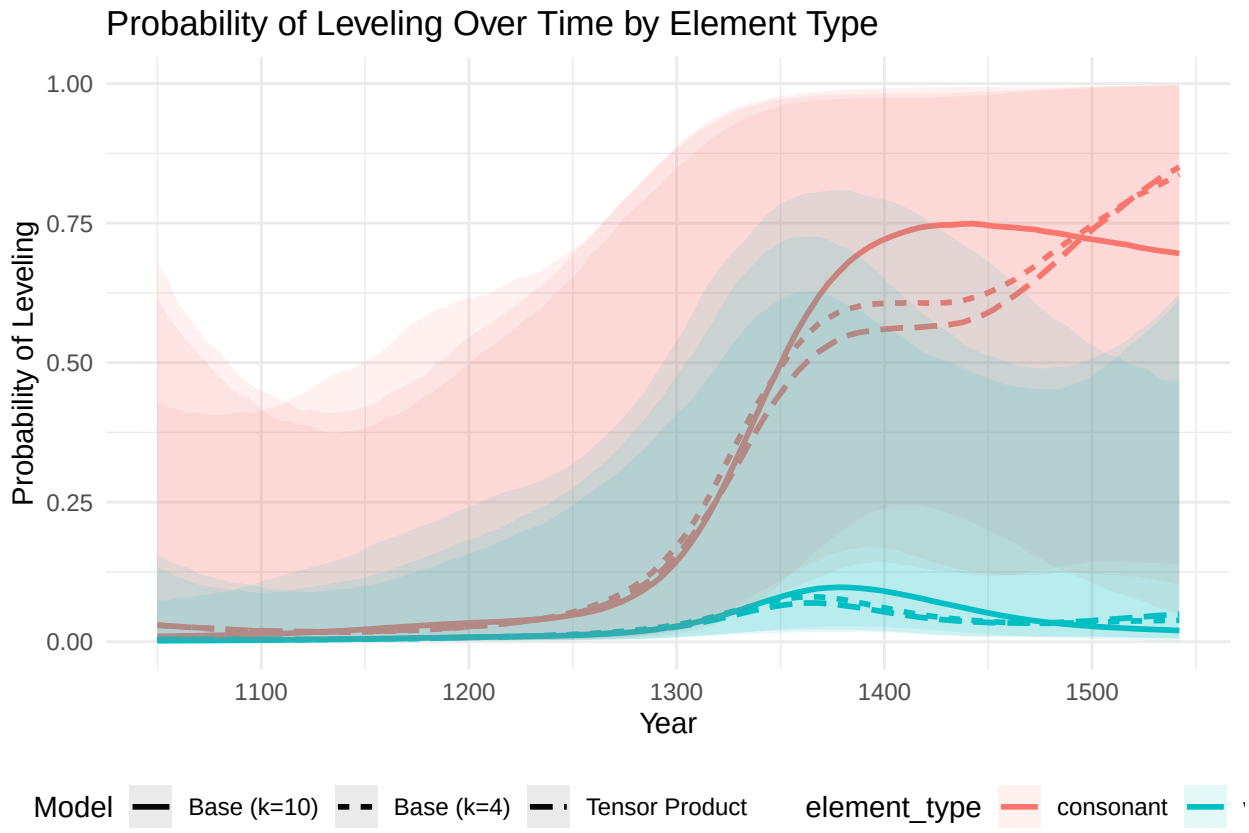
Time and vowel/consonant dynamics

```
# Extract conditional effects for date:element_type from all models
get_ce_data <- function(model, model_name) {
  ce <- conditional_effects(model, effects = "date:element_type")
```

```
  ce$`date:element_type` %>% mutate(Model = model_name)
}

all_ce_data <- map2_dfr(models, names(models), get_ce_data)

ggplot(all_ce_data, aes(x = date, y = estimate__, color = element_type, fill = element_type, linetype =
  geom_line(size = 1) +
  geom_ribbon(aes(ymin = lower__, ymax = upper__), alpha = 0.1, color = NA) +
  labs(
    title = "Probability of Leveling Over Time by Element Type",
    x = "Year",
    y = "Probability of Leveling"
  ) +
  theme_minimal() +
  theme(legend.position = "bottom")
```



Paradigm cell effect

```
get_infl_ce <- function(model, model_name) {
  ce <- conditional_effects(model, effects = "std_infl:element_type")
  ce$`std_infl:element_type` %>% mutate(Model = model_name)
}
```

```r
all_infl_ce <- map2_dfr(models, names(models), get_infl_ce)

ggplot(all_infl_ce, aes(x = std_infl, y = estimate__, color = Model, shape = element_type)) +
  geom_point(position = position_dodge(width = 0.4), size = 3) +
  geom_errorbar(aes(ymin = lower__, ymax = upper__), width = 0.2, position = position_dodge(width = 0.4
  labs(
    title = "Effect of Inflectional Context and Element Type on Leveling",
    x = "Inflectional Context",
    y = "Probability of Leveling"
  ) +
  theme_minimal() +
  theme(legend.position = "bottom")
```